SY B.Tech. - ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

PRACTICAL ASSIGNMENT-7

1. Student Details

StudentName	Homesh Sawarkar
PRN	202501110069
Division	A
Batch	A-2
Roll No.	202501110069
Date ofSubmission	24-10-2026

2. Practical Details

Practical No.	7
Practical Title	Implementation of Polymorphism in C++ (Function Overloading, Operator Overloading and Virtual Functions)
Date Performed	
Faculty Name	Dr. Khushal Khairnar

3. Problem Statement

Writea C++programtodemonstrate **polymorphism**. The program should show compile time polymorphism using **function overloading** and **operator overloading**, and run time polymorphism using a **virtual function** (function overriding) called through a base class pointer.

4. Objectives

1. To understand the meaning of polymorphism (one name, many forms).
2. To understand compile time and run time polymorphism.
3. To implement function overloading and operator overloading.
4. To implement function overriding using virtual functions and a base class pointer.
5. To understand the difference between overloading and overriding.

5. Expected Learning Outcomes

- Writemore thanonefunctionwiththe samename but different parameters.
- Change the meaning of an operator for a user-defined class.
- Use the keyword virtual to call the correct function of a derived class through a base class pointer.
- Identify whether a given example is compile time or run time polymorphism.

6. Theory / Concept

Polymorphism: The word polymorphism means 'many forms'. In C++ it means that the same name (function or operator) can behave in different ways in different situations. For example, a person can be a student, a son and a friend at the same time, and behaves differently in each role. Polymorphism is of two types: compile time and run time.

1. Compile time polymorphism (static binding): The compiler decides which function to call while compiling the program, so it is fast. It is achieved by function overloading and operator overloading.

Function overloading: Many functions can have the same name in the same class if their parameters are different (different number of parameters or different types). The compiler chooses the correct one by looking at the arguments in the call. The return type alone cannot be used to differentiate the functions. Example: add(int, int), add(int, int, int) and add(float, float).

Operator overloading: It gives a special meaning to an existing operator (like +, -, ==) for objects of a user-defined class. It is done with the keyword operator, for example Complex operator+(Complex c). Then c3 = c1 + c2; works for objects like it works for numbers. The meaning of the operator for built-in types does not change, and operators like :: (scope), . (dot) and ?: cannot be overloaded.

2. Run time polymorphism (dynamic binding): The function to be called is decided while the program is running, according to the type of the object that the pointer points to. It is achieved by function overriding using virtual functions.

Function overriding: When a derived class writes its own version of a function that already exists in the base class with the same name and same parameters, it is called overriding. If the base class function is declared virtual, then a base class pointer pointing to a derived class object calls the derived class version. Without virtual, the base class version would be called.

Pure virtual function and abstract class: A virtual function with no body, written as virtual void sound() = 0;, is a pure virtual function. A class that has at least one pure virtual function is an abstract class; its object cannot be created and the derived classes must override the function.

Point	Compile time	Run time
Also called	Static binding / early binding	Dynamic binding / late binding
Achieved by	Function and operator overloading	Virtual functions (overriding)
Decided	By the compiler	While the program runs
Speed	Faster	Slightly slower

7. Algorithm / Pseudocode

1. START
2. Create class Calculator with three functions named add(): add(int, int), add(int, int, int) and add(float, float).
3. Create object calc and call add(2, 3), add(1, 2, 3) and add(1.5, 2.5); display the results.
4. Create class Complex with real and imag, and write operator+ which adds the real parts and the imaginary parts.
5. Read two complex numbers c1 and c2 from the user, calculate c3 = c1 + c2 and display c3.
6. Create class Animal with a virtual function sound(); create classes Dog and Cat derived from Animal, each overriding sound().
7. Create a base class pointer ptr of type Animal.
8. Make ptr point to a Dog object and call ptr->sound(); then make it point to a Cat object and call ptr->sound().
9. STOP

08. Technology / Tools / Libraries Used

Technology / Tool	Purpose
Programming Language	C++
IDE / Platform	Visual Studio Code (VS Code)
Libraries / Packages	iostream

09. Implementation / GitHub Repository

Particular	Detail
GitHub Repository URL	Your paragraph text
Repository Name	Assignment 7
Branch Used	main
Commit ID / Latest Commit	Add file via upload
Dateof Last Commit	24-10-2026

10. Test Cases / Input and Output

Test Case	Input	ExpectedOutput	Actual Output	Status
1	2 3 4 5	add(2,3) = 5; add(1,2,3) = 6; add(1.5,2.5) = 4; Sum = 6 + 8i; Dog says: Bhow Bhow; Cat says: Meow Meow		
2	1 1 2 2	add(2,3) = 5; add(1,2,3) = 6; add(1.5,2.5) = 4; Sum = 3 + 3i; Dog says: Bhow Bhow; Cat says: Meow Meow		

```
PS C:\Users\Homesh> cd "c:\Users\Homesh\" ; if ($?) { g++ assi7.cpp -o assi7 } ; if ($?) { .\assi7 }
--- 1. Function Overloading ---
add(2, 3)       = 5
add(1, 2, 3)    = 6
add(1.5, 2.5)   = 4

--- 2. Operator Overloading ---
Enter real and imaginary part of first number: 2 3
Enter real and imaginary part of second number: 4 5
Sum = 6 + 8i

--- 3. Virtual Function ---
Dog says: Bhow Bhow
Cat says: Meow Meow
PS C:\Users\Homesh>
```

11. Results / Observations

The C++ program wasimplementedin Visual Studio Code. In function overloading, the compiler selected the correct add() function by looking at the number and type of arguments. In operator overloading, the + operator added two Complex objects directly (c3 = c1 + c2). In run time polymorphism, the same call ptr->sound() printed different messages for Dog and Cat because sound() is virtual and the function was chosen according to the object that the base class pointer pointed to. Both test cases gave the expected output.

12. Conclusion

Thepractical successfully demonstrated polymorphism in C++. Function overloading and operator overloading are examples of compile time polymorphism, and virtual functions are an example of run time polymorphism. Polymorphism makes the program easier to read, easier to extend and more flexible.

13. Assignment Questions

Q1.What is polymorphism? Explainits types.

Answer: Polymorphism means 'many forms': one name can work in different ways. It has two types. Compile time polymorphism is decided by the compiler and is achieved by function overloading and operator overloading. Run time polymorphism is decided while the program is running and is achieved by virtual functions (function overriding).

Q2. What is the difference between function overloading and function overriding?

Answer: In function overloading, many functions with the same name but different parameters are written in the same class, and it is resolved at compile time. In function overriding, a derived class writes a new version of a base class function with the same name and same parameters, and it is resolved at run time when the function is virtual. Overloading happens within one class, while overriding needs inheritance.

Q3. What is a virtual function? Why is it needed?

Answer: A virtual function is a base class function declared with the keyword virtual. It is needed so that when a base class pointer points to a derived class object, the derived class version of the function is called. Without virtual, the pointer always calls the base class version because the pointer type decides the function.

Q4. What is operator overloading? Which operators cannot be overloaded?

Answer: Operator overloading gives a new meaning to an operator for objects of a user-defined class using the keyword operator, for example operator+ to add two Complex numbers. The operators that cannot be overloaded are the scope resolution operator (::), the member access operator (.), the pointer to member operator (.*), the conditional operator (?:) and sizeof.

14. Student Declaration

Ihereby declarethatthe abovepractical work has been completed by me and that the implementation, analysis, and answers are my own work

Student Name	PRN	Signature
Homesh Sawarkar	202501110069	Homesh A.S